МИНИСТЕРСТВО ЦИФРОВЫХ ТЕХНОЛОГИЙ РЕСПУБЛИКИ УЗБЕКИСТАН
ТАШКЕНТСКИЙ УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
имени Мухаммада ал-Хоразмий

Дисциплина: «Введение в программную инженерию»

ПРАКТИЧЕСКАЯ РАБОТА № 4

Человеческий фактор: формирование команды проекта

Вариант 19: разработка АРМ администратора салона красоты

Выполнил: **Тожикулов Асилбек**
Группа: **M320-24**
Вариант: **19**
Принял: _____

Ташкент, 2026

1. Тема практической работы

Человеческий фактор: формирование команды проекта.

2. Цель работы и краткое описание проекта

Цель работы: закрепление знаний по теоретическим сведениям. Научиться формировать команду программного проекта: определять роли и компетенции, распределять ответственность с помощью матрицы RACI, выбирать структуру команды, планировать коммуникации и снижать риски человеческого фактора.

Вариант 19: разработка АРМ администратора салона красоты.

Проект BeautyDesk – веб-приложение для администраторов сети из трёх салонов красоты: база клиентов, график смен мастеров, журнал записи без накладок, касса с Payme и Click, напоминания клиентам, учёт материалов и расчёт зарплаты мастеров. По результатам предыдущих работ: ТЗ содержит 10 функций (ПР 1), выбрана инкрементная модель с тремя инкрементами и сроком декабрь 2026 – март 2027; бэклог из 15 историй (94 SP) ведётся по Scrum двухнедельными спринтами (ПР 2); первый прототип журнала записи оценён в 137 ч, принято решение go (ПР 3). Оценка ресурсов из ПР 3 показала, что проекту нужны backend-, frontend- и тестовые компетенции примерно в равном объёме, представитель заказчика с регулярным участием и человек, отвечающий за процесс.

3. Роли, обязанности и компетенции

Для проекта определены 6 ролей. Распределение по условным участникам сделано с учётом оценки ресурсов из ПР 3: на прототип требовалось 47 ч backend-работ, 55 ч frontend/UX и 35 ч тестирования, поэтому каждую из этих областей закрывает отдельный участник с полной занятостью.

Таблица 1 – Роли, обязанности и компетенции команды

Роль	Обязанности в проекте	Профессиональные компетенции	Личные качества	Участник
Спонсор (владелец сети)	утверждает бюджет и ТЗ, принимает инкременты, решает спорные вопросы приоритетов между салонами	понимание бизнеса салонов, финансовое планирование	решительность, доверие к команде	владелец сети салонов
Владелец продукта	ведёт и упорядочивает бэклог, пишет истории и критерии приёмки, принимает работу, собирает отзывы администраторов	знание процессов салона, формулирование требований, приоритизация (MoSCoW)	ответственность, доступность для команды, умение говорить «нет»	Нигора Рахимова

Роль	Обязанности в проекте	Профессиональные компетенции	Личные качества	Участник
Scrum-мастер / руководитель процесса	проводит события Scrum, следит за сроками и ёмкостью, устраняет препятствия, ведёт доску Trello и план коммуникаций	Scrum Guide, планирование, оценка трудозатрат, управление рисками	коммуникабельность, терпение, умение сглаживать конфликты	Асилбек Тожикулов
Backend-разработчик	архитектура, база данных, API, проверка пересечений записей, интеграции с Payme, Click, кассой, SMS	Python/Django, PostgreSQL, Redis, REST, транзакции и блокировки	внимательность к деталям, самостоятельность	Бахтиёр Юсупов
Frontend-разработчик / UX	макеты экранов, интерфейс АРМ, календарь записей, адаптивная версия, локализация RU/UZ	React, TypeScript, HTML/CSS, Figma, основы UX	эмпатия к пользователю, аккуратность	Мадина Алиева
Аналитик-тестировщик	уточнение требований с администраторами, тестовые данные, автотесты, проверка критериев приёмки, документация пользователя	анализ требований, тест-дизайн, pytest, Playwright, SQL	дотошность, умение задавать вопросы, коммуникабельность	Шахзод Каримов

Небольшая команда требует совмещения: Scrum-мастер дополнительно ведёт план коммуникаций и риски, аналитик-тестировщик отвечает и за обучение администраторов при внедрении. Ключевые компетенции продублированы хотя бы частично – frontend-разработчик знает основы Django, backend-разработчик может проверять тесты, – это снижает риск «единственного носителя знаний» (см. раздел 7).

4. Матрица ответственности RACI

Обозначения: **R** (Responsible) – исполняет работу; **A** (Accountable) – отвечает за результат и утверждает его, в строке ровно один; **C** (Consulted) – консультирует до выполнения; **I** (Informed) – информируется о результате. **A/R** – участник одновременно отвечает за результат и выполняет работу.

Таблица 2 – Матрица RACI для 10 основных работ проекта

Работа	Спонсор	Владелец продукта	Scrum-мастер	Backend	Frontend / UX	Аналитик-тестировщик
1. Утверждение ТЗ и бюджета	A	R	C	I	I	C
2. Ведение и приоритизация бэклога	C	A	C	C	C	R
3. Проектирование архитектуры	I	C	C	A	R	C
4. Макеты интерфейса и прототип	I	A	I	C	R	C
5. Разработка серверной части и API	I	I	I	A/R	C	C
6. Разработка интерфейса АРМ	I	C	I	C	A/R	C
7. Интеграции Payme/Click, касса, SMS	C	I	C	A	C	R
8. Тестирование и приёмка историй	I	A	C	C	C	R
9. Организация событий Scrum	I	C	A	R	R	R
10. Внедрение и обучение администраторов	C	A	R	I	C	R

Проверка матрицы. Матрица оформлена в виде таблицы, и правила проверены подсчётом по строкам и столбцам.

Таблица 3 – Проверка правил матрицы RACI по строкам

Правило	Результат	Нарушений	Вывод
В каждой строке ровно одна A	10 строк из 10	0	выполнено
В каждой строке хотя бы одна R (или A/R)	10 строк из 10	0	выполнено
У каждой работы есть исполнитель из команды разработки или заказчика	10 из 10	0	выполнено

Таблица 4 – Проверка загрузки участников по столбцам

Участник	A	R	A + R (работ)	Загрузка
Спонсор	1	0	1	низкая, соответствует роли
Владелец продукта	4	1	5	умеренная
Scrum-мастер	1	1	2	низкая (основная работа – С)
Backend	3	2	5	умеренная
Frontend / UX	1	4	5	умеренная
Аналитик-тестировщик	0	5	5	умеренная

Ни у одного участника нет больше 5 работ с ответственностью A или R из 10, нагрузка распределена равномерно между тремя разработчиками и владельцем продукта. В первоначальном варианте матрицы backend-разработчик был отмечен A и R в работах 3, 5 и 7 и одновременно R в тестировании интеграций – 6 работ; исполнение интеграционного тестирования (работа 7) передано аналитику-тестировщику, а исполнение части архитектуры (схемы клиента) – frontend-разработчику, после чего перегрузка устранена.

5. Выбор структуры команды

5.1. Оценка проекта по факторам выбора структуры

Л. Константин выделяет четыре организационные парадигмы: **закрытую** (традиционная иерархия, решения сверху), **случайную** (свободные творческие группы с минимальным управлением), **открытую** (сочетание управления и горизонтальной кооперации, решения принимаются коллективно) и **синхронную** (части работы выполняются параллельно и независимо при общем видении). Гибкая самоорганизующаяся команда в Scrum по сути реализует открытую парадигму. Проект оценён по типовым факторам выбора структуры.

Таблица 5 – Оценка проекта по факторам выбора структуры

Фактор	Значение для BeautyDesk	Что подходит
Сложность задачи	средняя	открытая или самоорганизующаяся – нужна совместная работа над журналом записи и интеграциями
Размер программы и команды	небольшие: 5–6 человек	плоская структура без промежуточных руководителей
Срок жизни команды	4 месяца + сопровождение	команда сохраняется на весь проект, есть время сработаться

Фактор	Значение для BeautyDesk	Что подходит
Модульность задачи	средняя: модули Ф1–Ф9 связаны журналом записи	частичная независимость, но с частым согласованием
Требуемая надёжность	высокая для журнала и кассы	коллективные ревью и тестирование
Жёсткость сроков	умеренная, поэтапная сдача	итеративная работа спринтами
Требуемая коммуникабельность	высокая: заказчик и пользователи участвуют постоянно	открытые горизонтальные связи
Ясность требований	основные ясны, детали уточняются	гибкая команда, быстро реагирующая на изменения

5.2. Выбор и обоснование

Таблица 6 – Сравнение структур команды

Структура	Оценка (1–5)	Комментарий
Закрытая (иерархическая)	2	хороша для крупных проектов с ясными требованиями; для 5 человек лишняя иерархия замедлит реакцию на отзывы салона
Случайная	1	подходит для исследовательских задач; не обеспечит дисциплину сроков и надёжность кассы
Синхронная	3	модули можно делать параллельно, но журнал записи требует постоянного согласования между backend и frontend
Открытая / гибкая самоорганизующаяся (Scrum)	5	соответствует размеру, сложности и требованию частой обратной связи; уже используется процесс Scrum из ПР 2

Выбрана гибкая самоорганизующаяся команда (открытая парадигма). Над командой разработки есть только две роли, задающие направление: спонсор (цель и бюджет) и владелец продукта (что делать и в каком порядке). Внутри команды нет начальника: разработчики сами распределяют задачи на планировании спринта, решения по архитектуре принимаются коллективно, а Scrum-мастер выступает фасилитатором, а не руководителем. Такая структура соответствует малому размеру команды, средней сложности задачи, частым уточнениям требований и инкрементной модели процесса.

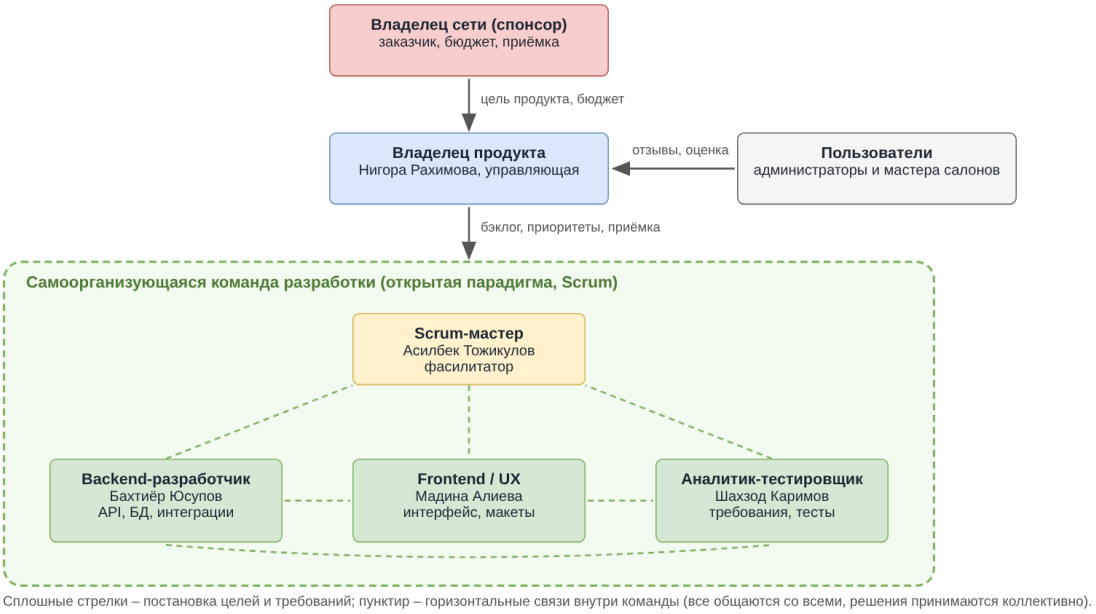


Рисунок 1 – Схема структуры команды проекта BeautyDesk (draw.io)

6. План коммуникаций и инструменты совместной работы

Таблица 7 – План коммуникаций

Мероприятие / канал	Периодичность	Участники	Цель	Инструмент
Ежедневный Scrum	ежедневно, 15 мин, 10:00	разработчики, Scrum-мастер	синхронизация, препятствия	Google Meet, доска Trello
Планирование спринта	раз в 2 недели, до 4 ч	вся Scrum-команда	цель и бэклог спринта	Trello, Google Таблицы (ёмкость)
Обзор спринта (демо)	раз в 2 недели, 2 ч	команда, владелец продукта, администраторы, спонсор (по желанию)	показ инкремента, отзывы	тестовый сервер, Google Meet
Ретроспектива	раз в 2 недели, 1,5 ч	Scrum-команда	улучшение процесса	Miro (доска ретро)
Отчёт спонсору	раз в 2 недели	Scrum-мастер, владелец продукта → спонсор	прогресс, бюджет, риски	Google Docs (1 страница), электронная почта
Рабочий чат команды	постоянно	вся команда	оперативные вопросы	Telegram (группа проекта)
Ревью кода	по каждому изменению	разработчики	качество кода, обмен знаниями	GitHub Pull Requests

Мероприятие / канал	Периодичность	Участники	Цель	Инструмент
Сбор отзывов из салонов	еженедельно в период внедрения	аналитик-тестировщик, администраторы	замечания и ошибки	Google Формы, Telegram
База знаний проекта	постоянно, обновление по итогам спринта	вся команда	ТЗ, решения, инструкции	Google Диск / Notion

Инструменты выбраны по принципу «бесплатно, знакомо команде и заказчику, работает в Узбекистане»: Telegram – основной мессенджер в стране, им пользуются и администраторы салона; Trello уже содержит бэклог (ПР 2); Google Workspace бесплатен и удобен для совместного редактирования таблиц и документов; GitHub используется для кода и ревью. Для каждого канала определено, кто отвечает за его ведение: протоколы событий Scrum и отчёт спонсору готовит Scrum-мастер, базу знаний обновляют все по итогам спринта.

7. Риски человеческого фактора

Вероятность и влияние оценены по шкале 1–5, приоритет = вероятность × влияние (высокий – 12 и выше, средний – 6–11, низкий – до 5).

Таблица 8 – Риски человеческого фактора

Риск	Вер.	Вл.	Приор.	Меры по снижению	Ответственный
Уход или болезнь единственного backend-разработчика (носитель знаний о журнале записи)	3	5	15	ревью кода всеми разработчиками, документирование архитектуры, frontend-разработчик осваивает Django, резерв подрядчика	Scrum-мастер
Владелец продукта занята работой салона и не успевает отвечать команде	4	4	16	фиксированные «часы владельца продукта» 2 раза в неделю, заместитель – старший администратор, истории готовятся заранее	спонсор
Сопротивление администраторов переходу с бумажного журнала	3	4	12	участие администраторов в демо с первого спринта, обучение, внедрение сначала в одном салоне, бумажный журнал как резерв на 2 недели	владелец продукта

Риск	Вер.	Вл.	Приор.	Меры по снижению	Ответственный
Перегрузка и выгорание к концу проекта (сжатые сроки)	3	3	9	планирование не более 90 % ёмкости, контроль загрузки по RACI, ретроспективы, запрет на сверхурочные как норму	Scrum-мастер
Недопонимание требований между командой и салоном (разные термины)	3	3	9	глоссарий терминов, критерии приёмки к каждой истории, аналитик на демо уточняет термины	аналитик-тестировщик
Конфликт в команде из-за распределения задач или оценок	2	3	6	коллективное планирование и покер оценки, правила работы команды, разбор на ретроспективе	Scrum-мастер
Недостаток опыта команды с API Payme, Click и онлайн-кассой	3	3	9	раннее изучение тестовых сред, консультация поставщика, время на исследование в спринте	backend-разработчик

Наибольший приоритет у двух рисков – недоступности владельца продукта (16) и зависимости от одного backend-разработчика (15); меры по ним начинают выполняться уже в первом спринте. Риски пересматриваются на каждой ретроспективе, а их состояние включается в отчёт спонсору.

8. Вывод о проделанной работе

Для проекта «АРМ администратора салона красоты» сформирована команда из 6 ролей: спонсор, владелец продукта, Scrum-мастер, backend- и frontend-разработчики и аналитик-тестировщик. Для каждой роли описаны обязанности, профессиональные компетенции и личные качества; роли распределены между условными участниками с учётом оценки ресурсов из ПР 3.

Построена матрица RACI для 10 основных работ проекта. Проверка показала, что в каждой строке ровно одна A и хотя бы одна R, а перегруженных участников нет: максимум 5 работ с ответственностью A или R у каждого; найденная в первом варианте перегрузка backend-разработчика устранена перераспределением работ.

По факторам выбора структуры (небольшой размер, средняя сложность, частые уточнения требований, высокая потребность в коммуникациях) выбрана гибкая самоорганизующаяся команда в рамках открытой парадигмы Константина, построена её схема. Составлен план из 9 мероприятий и каналов коммуникации с инструментами (Telegram, Trello, Google Workspace, GitHub, Miro). Выявлены 7 рисков человеческого фактора; самые значимые – недоступность владельца продукта и зависимость от одного backend-разработчика, для них определены меры и ответственные.